```
iex> [first, second | rest] = [1, 2, 3, 4]
iex> {first, second}
{1, 2}

iex> [first, second] ++ rest = [1, 2, 3, 4]
iex> {first, second}
{1, 2}
```

The | operator is used inside of the list brackets to represent the beginning of the list. One or more elements can be matched at a time.

The ++ operator is used to capture the concatenation of two lists. It's less common to see this syntax, though.

One thing you'll notice from these match clauses is that they behave exactly like their function versions. This makes the syntax intuitive to use. If you can use <> or [|] in your code, then you can use it in a pattern match clause. For example:

```
iex> [1, 2] ++ [3, 4]
[1, 2, 3, 4]

iex> [1, 2] ++ rest = [1, 2, 3]
iex> rest
[3]
```

In this example, we're able to use the ++ operator as a function (left ++ right) and as a match.

Pattern matching with maps is also intuitive, as in the following example:

```
iex> %{a: a} = %{a: 1, b: 2}
iex> a
1

iex> %{a: 1, b: nil} = %{a: 1, b: 2}
** (MatchError) no match of right hand side value: %{a: 1, b: 2}

iex> %{list: [%{a: ["a"]}, %{b: [b]}]} = %{list: [%{a: ["a"]}, %{b: ["b"]}]}
iex> b
"b"
```

This example is a bit dense, but it shows you that the complexity of the match clause isn't limited as long as it uses valid syntax.

Map matching behaves differently than lists because maps don't have to perfectly match. In the first example, the left side doesn't mention the `b` key at all. Maps are loosely matched when the key isn't specified. This is useful in practice because you often extract a few keys of a map. Here's a simple example to demonstrate this: